
Les files et les piles

Plusieurs phénomènes peuvent être modélisés en utilisant la notion de file ou la notion de pile. Ces deux structures qu'on retrouve souvent en programmation sont des cas particuliers des listes.

Les files

- Une file est une liste particulière dans laquelle on peut ajouter des éléments uniquement à l'extrémité B (la queue de la file) et en retirer à l'extrémité A (la tête de la file). L'exemple classique est une file d'attente au supermarché.
- On qualifie ces listes de FIFO (First-In, First-Out) pour exprimer la dynamique de premier entré, premier servi.

En Python, on modélisera une file en créant d'abord une liste vide puis en y ajoutant des éléments à la fin et en y retirant des éléments au début. Pour modéliser l'entrée et la sortie d'éléments dans une file FILE1 implémentée par une liste nous utiliserons les opérations suivantes:

- FILE1.append(elem)
pour ajouter un élément
- sortie = FILE1.pop(0)
pour retirer un élément

Par exemple, si nous voulions programmer le flot de 5 personnes qui arrivent à l'unique caisse ouverte au supermarché nous pourrions avoir ceci:

Opération	Description
FILE1 = []	Ouverture de la caisse
FILE1.append("Annie")	Annie se présente à la caisse et commence à se faire servir.
FILE1.append("Bernard")	Bernard se présente à la caisse et attend derrière Annie.
FILE1.append("Carole")	Carole se présente à la caisse et attend derrière Bernard.
FILE1.pop(0)	Annie quitte la caisse. Bernard se fait servir.
FILE1.append("Denis")	Denis se présente à la caisse et attend derrière Carole.
FILE1.pop(0)	Bernard quitte la caisse. Carole se fait servir.
FILE1.pop(0)	Carole quitte la caisse. Denis se fait servir.
FILE1.append("Élise")	Élise se présente à la caisse et attend derrière Denis.
FILE1.pop(0)	Denis quitte la caisse. Élise se fait servir.
FILE1.pop(0)	Élise quitte la caisse. La file est vide, la caissière peut respirer.

Les piles

- Une pile est une liste particulière dans laquelle on peut ajouter et retirer des éléments uniquement à une extrémité. L'exemple classique est une pile de tâches à exécuter dans laquelle le dernier item ajouté sur la pile est toujours le plus urgent et c'est celui qui devrait être traité en priorité.
- On qualifie ces listes de LIFO (Last-In, First-Out) pour exprimer la dynamique de dernier entré, premier traité.

En Python, on modélisera une pile en créant d'abord une liste vide puis en y ajoutant des éléments à la fin et en y retirant également des éléments à la fin. Pour modéliser l'entrée et la sortie d'éléments dans une pile PILE1 implémentée par une liste nous utiliserons les opérations suivantes:

- `PILE1.append(elem)`
pour ajouter un élément
- `sortie = PILE1.pop()`
pour retirer un élément

Les tuples

Les tuples sont pratiquement identiques aux listes sauf qu'un tuple ne peut pas être modifié après sa création. Le tuple est donc une séquence non modifiable.

Construction d'un tuple

- Un tuple est défini simplement en utilisant l'opérateur d'affectation =
- On délimite les éléments avec des parenthèses: (et)
- On sépare les éléments du tuple avec des virgules
- Tout comme pour les listes on peut également utiliser la fonction range. Cependant, pour générer un tuple la séquence produite par la fonction range doit être passée à la fonction de conversion en tuple.

```
>>> maListe = [*range(5)]
>>> print(maListe)
[0, 1, 2, 3, 4]
>>> monTuple = tuple(maListe)
>>> print(monTuple)
(0, 1, 2, 3, 4)
```

Exemples (idem aux listes en remplaçant les [] par des ()):

- tuple1 = (2, 7, 8, -6)
Initialise un tuple de 4 éléments qui sont des nombres entiers
- tuple2 = (2, 1.6, "Bonjour toi", 3, 6, 9)
Initialise un tuple de 6 éléments: 4 nombres entiers (int), 1 nombre décimal (float) et 1 chaîne de caractères (str)
- tuple3 = ()
Initialise un tuple vide. Inutile en pratique car le tuple étant non modifiable par définition il restera toujours vide.
- tuple4 = (1, 2, (5, 6, 7), 3, 4)
Initialise un tuple de 5 éléments: 4 nombres entiers et le 3^{ème} élément est lui-même un tuple de 3 éléments.
Ainsi:

```
type(tuple4[2]) retourne "tuple"
tuple4[2] retourne le tuple (5, 6, 7)
tuple4[2][1] retourne le nombre 6
```

-
- `tuple5 = ( (1, 2, 3), (4, -1, 9), (-2, 5, 0), (3, 2, 7) )`
Initialise un tuple de 4 éléments, chaque élément du tuple étant lui-même un tuple de 3 éléments. On a donc un tuple de tuples. On peut l'illustrer par la matrice suivante:

1	2	3
4	-1	9
-2	5	0
3	2	7

`tuple5[2][1]` retourne la valeur 5 car le nombre 5 correspond au 2ème élément (indice 1) du 3^{ème} élément (indice 2) du tuple "tuple5".

- `tuple6 = tuple(range(1, 100, 2))`
Initialise un tuple de 50 éléments avec les nombres impairs de 1 à 99 inclusivement.

Attention: ne pas oublier l'opérateur de dépaquetage '*' si on veut un type 'tuple' et non pas un type 'range'

```
>>> maListe = [*range(5)]
>>> print(maListe)
[0, 1, 2, 3, 4]
>>> monTuple = tuple(maListe)
>>> print(monTuple)
(0, 1, 2, 3, 4)
```

Opérations, fonctions et méthodes sur les séquences : listes, tuples et chaînes

Plusieurs traitements de base sont définis sur les séquences. Étant donné les séquences suivantes qui contiennent respectivement 5 éléments et 4 éléments:

SEQ1 : 3, 6, 9, 2, 2
SEQ2 : 0, 3, 9, 1

Les opérations suivantes sont définies:

Opérateur	Opération (définition)	Exemples
in not in	Vérifie si un élément fait partie (on ne fait pas partie) de la séquence. Retourne une valeur booléenne True ou False.	2 in SEQ1 retourne True 4 in SEQ2 retourne False 7 not in SEQ1 retourne True
+	Concaténation : Juxtapose les éléments de chaque séquence.	SEQ2 + SEQ1 retourne la séquence: 0, 3, 9, 1, 3, 6, 9, 2, 2
*	Concaténation multiple.	SEQ2*2 retourne la séquence: 0, 3, 9, 1, 0, 3, 9, 1
[i]	Accède au i-ème élément d'une séquence. La numérotation des indices commençant à 0 pour accéder au premier élément.	SEQ1[2] retourne la valeur 9
[i:j]	Accède à la sous-séquence commençant à l'indice i et se terminant à l'indice j-1.	SEQ1[1:3] retourne la séquence: 6, 9
[i:]	Accède à la sous-séquence commençant à l'indice i et se terminant à la fin de la séquence.	SEQ1[2:] retourne la séquence: 9, 2, 2
[:j]	Accède à la sous-séquence commençant à l'indice 0 et se terminant à l'indice j-1.	SEQ1[:1] retourne la séquence: 3, 6

Les fonctions suivantes sont définies:

Fonction	Définition	Exemples
len()	Retourne le nombre d'éléments d'une séquence.	len(SEQ1) retourne 5 len(SEQ2) retourne 4
list(sequence)	Retourne une liste construite avec chaque élément de la séquence en paramètre. Ainsi, si la séquence est un tuple, cette fonction permet de construire une liste avec les mêmes valeurs que le tuple.	liste1 = list(tuple1)
tuple(sequence)	Retourne un tuple construit avec chaque élément de la séquence en paramètre. Ainsi, si la séquence est une liste, cette fonction permet de construire un tuple avec les mêmes valeurs que la liste.	tuple2 = tuple(liste2)
set(sequence)	Retourne un ensemble construit à partir des éléments de la séquence en paramètre. Ainsi, si la séquence est une liste ou un tuple, cette fonction permet de construire un ensemble de <u>valeurs distinctes</u> avec les valeurs de la liste ou du tuple.	ensemble1 = set(tuple1) ensemble2 = set(liste1)

Les méthodes suivantes sont définies:

Méthode	Définition	Exemples
index(elem)	Retourne la position de la première occurrence de l'élément "elem" dans la séquence. Attention, une erreur se produit si l'élément n'est pas dans la séquence.	SEQ1.index(2) retourne 3
count(elem)	Retourne le nombre d'occurrences de l'élément "elem" dans la séquence.	SEQ1.count(2) retourne 2 SEQ1.count(8) retourne 0

Opérations, fonctions et méthodes spécifiques aux listes

liste7 = ["Patrice", "Louise", "Martine", "Benoit"]

liste8 = [5, 3, -7, 4, 3, 2, 0]

liste9 = [5.0, 3.0, -7, 4, 3, 2.0, 0]

Les opérations suivantes sont définies et permettent de modifier ces listes:

Opérateur	Opération (définition)	Exemples
<code>...[i] = ...</code>	Remplacement du i-ème élément d'une liste. Le i-ème élément est remplacé par l'élément à la droite de l'opérateur d'affectation (=).	liste7[1] = "Camille". Après l'opération, la liste contiendra les 4 éléments: Patrice, Camille, Martine et Benoit.
<code>...[i:j] = ...</code>	Remplacement d'une sous-liste par chaque élément retrouvé dans un "itérable". Les éléments <i>i</i> à <i>j-1</i> sont remplacés par la séquence d'éléments de l'itérable à la droite de l'opérateur d'affectation (=).	liste7[1:3] = ["Luc", "Anne", "Carl"] Après l'opération, la liste contiendra les 5 éléments: Patrice, Luc, Anne, Carl, Benoit.
<code>del ... [i:j]</code>	Supprime les éléments <i>i</i> à <i>j-1</i> de la liste.	del liste7[1:3] Après l'opération, la liste contiendra les 2 éléments: Patrice, Benoit.

Les méthodes suivantes sont définies:

Méthode	Définition	Exemple
<code>append(elem)</code>	Ajoute l'élément à la fin de la liste.	liste7.append("Sylvie") Après l'opération, la liste contiendra les 5 éléments: Patrice, Louise, Martine, Benoit, Sylvie.
<code>extend(iterable)</code>	Ajoute tous les éléments de l'itérable à la fin de la liste.	liste7.extend(["Luc", "Marc"]) Après l'opération, la liste contiendra les 6 éléments: Patrice, Louise, Martine, Benoit, Luc, Marc.
<code>insert(indice, elem)</code>	Insère l'élément "elem" à la position i.	liste7.insert(1, "Daniel") Après l'opération, la liste contiendra les 5 éléments: Patrice, Daniel, Louise, Martine, Benoit.

Méthode	Définition	Exemple
<code>pop(indice)</code>	Supprime l'élément de la position <i>i</i> et retourne l'élément supprimé comme résultat. Si la méthode est appelée sans paramètre, c'est le dernier élément de la liste qui est supprimé et retourné.	<code>sup = liste7.pop(2)</code> Après l'opération, la liste contiendra les 3 éléments: Patrice, Louise, Benoit et la variable <code>sup</code> contiendra la valeur "Martine".
<code>remove(elem)</code>	Supprime la première occurrence de l'élément <i>elem</i> si celui-ci se trouve dans la liste. Si l'élément ne fait pas partie de la liste une erreur se produit.	<code>liste8.remove(3)</code> . Après l'opération, la liste contiendra les 6 éléments: 5, -7, 4, 3, 2, 0.
<code>reverse()</code>	Inversion des éléments de la liste.	<code>liste8.reverse()</code> Après l'opération, la liste contiendra les 7 éléments: 0, 2, 3, 4, -7, 3, 5.
<code>sort([cmp=fonctionCmp,] [key=fonctionCle,] [reverse=booléen])</code>	Tri des éléments de la liste. Il est possible de fournir 3 paramètres optionnels à la méthode pour contrôler le tri. Le premier permet de fournir une fonction de comparaison entre 2 éléments pour déterminer la relation d'ordre entre les 2. Le second permet de fournir une fonction pour déterminer la valeur à comparer lors du tri. Le troisième permet de préciser l'ordre du tri: croissant ou décroissant.	<code>liste8.sort()</code> Après l'opération, la liste contiendra les 7 éléments: -7, 0, 2, 3, 3, 4, 5. <code>liste8.sort(reverse=True)</code> Après l'opération, la liste contiendra les 7 éléments: 5, 4, 3, 3, 2, 0, -7. <code>liste8.sort(key=abs)</code> Après l'opération, la liste contiendra les 7 éléments: 0, 2, 3, 3, 4, 5, -7. Soit <code>ABC</code> une fonction de deux paramètres qui retourne la valeur -1 si le premier paramètre est un int et si le second est un float, 0 si les 2 paramètres sont des int ou des float et enfin 1 si le premier est un float et le second un int. <code>liste9.sort(cmp=ABC)</code> Après l'opération, la liste contiendra les 7 éléments: -7, 4, 3, 0, 5.0, 3.0, 2.0.